TurnoYa

Sistema de reservas en línea para una clínica pequeña

Desarrollo de la primera versión mediante Spec-Driven Development con Spec Kit

Este informe presenta la primera versión funcional de TurnoYa: el problema que resuelve, el proceso de desarrollo seguido, el producto entregado, la verificación realizada y los pasos recomendados para la siguiente iteración.

Ficha del proyecto	Detalle
Proyecto	TurnoYa – Sistema de reservas para una clínica pequeña
Versión	1.0 (primera versión funcional)
Fecha	29 de octubre de 2026
Estado	Entregada; lista para validación con la clínica
Metodología	Spec-Driven Development con Spec Kit 1.2.5
Repositorio	test-spec-kit · rama master · commit 7ee19d4

Control del documento

Versión	Fecha	Descripción
1.0	29/10/2026	Informe de entrega de la versión 1 de TurnoYa.

Contenido

1. Resumen ejecutivo 3
2. Contexto y alcance 4
3. Metodología 5
4. Desarrollo del proyecto 6
4.1 Especificación · 4.2 Clarificación 6
4.3 Plan técnico 7
4.4 Planificación de tareas · 4.5 Implementación 8
5. Producto entregado 9
5.3 Recorrido por la aplicación 10
6. Calidad y verificación 12
7. Limitaciones y próximos pasos 14
A. Anexo A · Evidencias del proceso SDD 15
B. Anexo B · Ejecución del proyecto 19
C. Anexo C · Inventario de artefactos 20

Convenciones

- **US1…US5:** historias de usuario de la especificación; **FR-xxx:** requisitos funcionales; **SC-xxx:** criterios de éxito.
- **Rutas de archivo** relativas a la raíz del repositorio (por ejemplo, specs/001-reservas-turnoya/spec.md).
- **Capturas:** las del proceso muestran la salida real de los comandos; las del producto, la aplicación en ejecución.

1. Resumen ejecutivo

1.1 Objetivo

La clínica gestiona hoy sus citas por teléfono y WhatsApp, lo que genera citas duplicadas, olvidos y llamadas perdidas. El objetivo de la versión 1 fue entregar una aplicación web en la que los pacientes reserven, reprogramen y cancelen sus citas en línea, y en la que recepción, médicos y administración gestionen la agenda desde un único lugar, aplicando de forma automática las reglas de negocio de la clínica.

1.2 Qué se hizo

- **Se definió el marco del proyecto** en una constitución con seis principios obligatorios (pruebas de reglas de negocio, validación, accesibilidad, simplicidad, manejo del tiempo y notificaciones fiables).
- **Se especificó el producto** en cinco historias de usuario priorizadas y 24 requisitos funcionales, y se resolvieron cinco ambigüedades clave antes de diseñar.
- **Se diseñó la solución** sin dependencias externas: Node.js 24, base de datos SQLite embebida y páginas renderizadas en el servidor.
- **Se planificaron y ejecutaron 49 tareas** escribiendo primero las pruebas y luego el código de cada regla.
- **Se verificó el resultado** con 104 pruebas automáticas y un recorrido completo de la aplicación con cada rol.

1.3 Resultados clave

5 / 5 historias de usuario implementadas	**24** requisitos funcionales cubiertos	**49 / 49** tareas completadas
104 pruebas automáticas, 0 fallos	**0** dependencias de terceros	**47** capturas de evidencia

1.4 Estado y recomendación

La versión 1 está completa respecto de su especificación y lista para una validación con el personal de la clínica. Antes de ponerla en producción se recomienda conectar un servidor de correo real (hoy el envío es simulado), medir con usuarios los criterios de tiempo de uso y realizar una prueba manual de accesibilidad. El detalle está en el capítulo 7.

2. Contexto y alcance

2.1 Usuarios y funciones

Rol	Funciones en la versión 1
Paciente	Registro e inicio de sesión con correo y contraseña; búsqueda de disponibilidad por especialidad, médico y fecha; reserva con confirmación por correo; reprogramación y cancelación hasta 24 h antes; recordatorio automático 24 h antes.
Recepción	Agenda diaria y semanal; creación y modificación de citas en nombre de pacientes (incluidos los atendidos por teléfono); registro de asistencia.
Médico	Consulta de su agenda diaria y semanal; bloqueo de horarios por vacaciones o reuniones.
Administrador	Gestión de especialidades, médicos, horarios de atención y feriados; bandeja de correos enviados.

2.2 Reglas de negocio

Regla	Aplicación
Duración de la cita	30 min en consulta general; 45 min en especialidades
Solapamiento	Un médico no puede tener dos citas que se solapen
Citas activas por paciente	Máximo 3
Anticipación mínima para reservar	2 horas (límite inclusivo)
Reprogramación y cancelación	Hasta 24 horas antes (límite inclusivo)
Inasistencias	3 consecutivas bloquean la reserva en línea; solo recepción puede agendar

2.3 Fuera de alcance

Pagos, historia clínica, telemedicina, aplicación móvil e integración con seguros quedaron fuera de la versión 1 por decisión de alcance.

3. Metodología

El proyecto se desarrolló con Spec-Driven Development (SDD): primero se fija qué se construye y por qué, y solo después cómo. Cada fase se ejecutó con el comando correspondiente de Spec Kit y produjo un artefacto versionado que sirve de entrada a la siguiente. Así, cada línea de código se puede rastrear hasta una tarea, y cada tarea hasta un requisito.

3.1 Fases ejecutadas

Fase	Propósito	Artefacto resultante
Constitución	Fijar principios obligatorios	.specify/memory/constitution.md
Especificación	Describir qué y por qué, sin tecnología	specs/…/spec.md
Clarificación	Resolver ambigüedades antes de diseñar	spec.md (Clarifications)
Plan técnico	Decidir tecnología y diseño	plan.md, research.md, data-model.md, contracts/
Tareas	Descomponer en trabajo ejecutable	tasks.md (49 tareas)
Implementación	Construir y verificar	src/, tests/, README.md

3.2 Principios de la constitución

N.º	Principio	Exigencia
I	Reglas de negocio con pruebas	Cada regla se prueba en su frontera y en su violación; la lógica no depende de HTTP ni del reloj real.
II	Validación en el borde	Entradas validadas y escapadas; contraseñas con hash y sal; control de rol y propiedad en el servidor.
III	Accesibilidad básica	Uso con teclado, HTML semántico, contraste AA, errores en texto junto al campo.
IV	Simplicidad	Toda dependencia se justifica; no se construye nada fuera del alcance.
V	Manejo explícito del tiempo	Instantes en UTC; decisiones en la zona horaria de la clínica; reloj inyectable.
VI	Notificaciones fiables	Correos registrados antes de enviarse, sin duplicados y con reintento ante fallos.

3.3 Entorno

Windows 11 con PowerShell, Node.js 24 y Spec Kit 1.2.5 (integración agy). Las evidencias del Anexo A muestran la salida real de los comandos ejecutados en cada fase.

4. Desarrollo del proyecto

4.1 Especificación

Se creó la funcionalidad 001-reservas-turnoya y se redactó la especificación en lenguaje de negocio. Las historias se priorizaron para que cada una aporte valor por sí sola y pueda probarse de forma independiente. La especificación superó en la primera revisión los 16 controles de calidad de Spec Kit.

ID	Historia de usuario	Prioridad	Criterio de prueba independiente
US1	El paciente reserva una cita en línea	P1	La cita queda registrada, el horario deja de ofrecerse y se envía confirmación.
US2	El paciente reprograma o cancela; recibe recordatorio	P2	Cambios solo hasta 24 h antes; un único recordatorio por cita.
US3	Recepción gestiona agenda y asistencia	P2	Tres inasistencias bloquean la reserva en línea del paciente.
US4	El médico consulta su agenda y bloquea horarios	P3	Un horario bloqueado deja de estar disponible.
US5	El administrador configura médicos y horarios	P3	Un médico nuevo con horario genera disponibilidad.

4.2 Clarificación

Antes de diseñar se resolvieron cinco puntos que la definición inicial dejaba abiertos. En cada caso se adoptó la opción recomendada por la herramienta y la decisión se incorporó a la especificación como requisito.

Tema	Decisión	Requisito
Zona horaria	Una única zona horaria configurable de la clínica para mostrar y evaluar todas las horas.	FR-018
Feriados	El administrador los registra; esos días no se ofrece disponibilidad.	FR-019
Bloqueo con citas activas	Se rechaza y se listan las citas en conflicto para que recepción las reprograme.	FR-020
Conteo de inasistencias	Cuentan las inasistencias consecutivas; una asistencia reinicia el conteo; las cancelaciones no cuentan.	FR-021
Atribuciones de recepción	Exenta del límite de 3 citas, de las 2 h y del bloqueo; sujeta a duración y no solapamiento.	FR-022

4. Desarrollo del proyecto (continuación)

4.3 Plan técnico

La tecnología se eligió en esta fase, priorizando la simplicidad exigida por la constitución. Las seis comprobaciones de la constitución se cumplieron antes y después del diseño, sin excepciones que justificar.

Aspecto	Decisión	Justificación
Plataforma	Node.js 24, servidor HTTP propio, HTML generado en el servidor	Sin dependencias externas; funciona sin JavaScript en el navegador.
Datos	SQLite embebido	Transaccional y sin servicios adicionales que mantener.
Integridad de la agenda	Transacción atómica e índice único por médico y horario	Impide citas duplicadas incluso con solicitudes simultáneas.
Tiempo	UTC en la base; zona de la clínica para las reglas	Permite probar con exactitud los límites de 2 h y 24 h.
Seguridad	Contraseñas con scrypt y sal, sesiones HttpOnly, protección CSRF	Mecanismos estándar de la plataforma.
Correo	Bandeja de salida con reintentos	Un fallo de envío no afecta la cita.
Recordatorios	Proceso interno cada minuto, sin duplicados	Resistente a reinicios del servidor.

Arquitectura

Capa	Responsabilidad
Dominio	Reglas de negocio y cálculo de disponibilidad. No accede a la base de datos ni a la red, por lo que se prueba de forma aislada.
Servicios	Operaciones de negocio en transacciones: reservar, reprogramar, cancelar, asistencia, bloqueos, administración, correos y recordatorios.
Web	Rutas por rol, control de sesión y permisos, validación de formularios y páginas accesibles.

Modelo de datos. Diez entidades: usuarios, pacientes, especialidades, médicos, horarios de atención, feriados, bloqueos, citas, bandeja de correos y sesiones. Una cita pasa por los estados activa, cancelada, asistió o no asistió.

4. Desarrollo del proyecto (continuación)

4.4 Planificación de tareas

El plan se descompuso en 49 tareas agrupadas por historia de usuario, de modo que cada historia pudiera completarse y probarse por separado. En cada historia, las tareas de prueba preceden a las de implementación.

Bloque	Tareas	Contenido
Preparación	T001–T003	Proyecto, configuración y exclusiones de control de versiones
Base común	T004–T015	Tiempo, base de datos, validación, plantillas, sesiones, correos, datos de demostración
US1 · Reserva (P1)	T016–T026	Reglas, disponibilidad, registro, reserva y pantallas del paciente
US2 · Cambios (P2)	T027–T033	Ventana de 24 h, reprogramación, cancelación y recordatorios
US3 · Recepción (P2)	T034–T038	Agenda, citas por recepción, asistencia y bloqueo por inasistencias
US4 · Médico (P3)	T039–T041	Agenda del médico y bloqueos de horario
US5 · Administración (P3)	T042–T044	Especialidades, médicos, horarios y feriados
Cierre	T045–T049	Accesibilidad, concurrencia, documentación y seguridad

4.5 Implementación

La implementación siguió el orden de prioridades. Para cada regla se escribió primero la prueba, se comprobó que fallaba y después se implementó hasta que pasara. Las 49 tareas se completaron.

- **Pruebas primero:** la primera ejecución falló porque el código aún no existía; tras implementar el dominio pasaron las 38 pruebas iniciales.
- **Verificación de las pruebas:** se alteró a propósito la regla de 24 h y fallaron exactamente las pruebas de esa regla; luego se restauró.
- **Validación en ejecución:** se recorrió la aplicación real con cada rol. Así se detectaron y corrigieron tres incidencias (sección 6.5).

5. Producto entregado

La versión 1 implementa la totalidad de las funcionalidades especificadas para los cuatro roles. La aplicación se ejecuta con un único comando, incluye datos de demostración y funciona en navegadores de escritorio y en teléfonos.

5.1 Funcionalidades

Funcionalidad	Historia	Estado	Observación
Registro e inicio de sesión	US1	Completa	Validación de correo y contraseña segura
Búsqueda de disponibilidad	US1	Completa	Por especialidad, médico y fecha
Reserva con confirmación por correo	US1	Completa	Envío simulado en v1 (ver capítulo 7)
Reprogramación y cancelación	US2	Completa	Hasta 24 h antes
Recordatorio 24 h antes	US2	Completa	Uno por cita, sin duplicados
Agenda diaria y semanal	US3, US4	Completa	Recepción ve todas; el médico, solo la suya
Citas en nombre de pacientes	US3	Completa	Incluye pacientes sin cuenta
Asistencia y bloqueo por inasistencias	US3	Completa	Tres consecutivas bloquean la reserva en línea
Bloqueo de horarios	US4	Completa	Rechazado si hay citas en el rango
Administración y feriados	US5	Completa	Especialidades, médicos, horarios y feriados

5.2 Seguridad y accesibilidad

- Contraseñas almacenadas con hash y sal; sesiones con cookies protegidas y caducidad.
- Protección contra falsificación de solicitudes en todos los formularios y control de permisos por rol en el servidor.
- Cabeceras de seguridad, escape de todo contenido mostrado y ningún dato personal en los registros del servidor.
- Páginas semánticas usables con teclado, contraste conforme a WCAG AA y diseño adaptable a teléfonos.

5. Producto entregado (continuación)

5.3 Recorrido por la aplicación

Las capturas siguientes se tomaron sobre la aplicación en ejecución con datos de demostración.

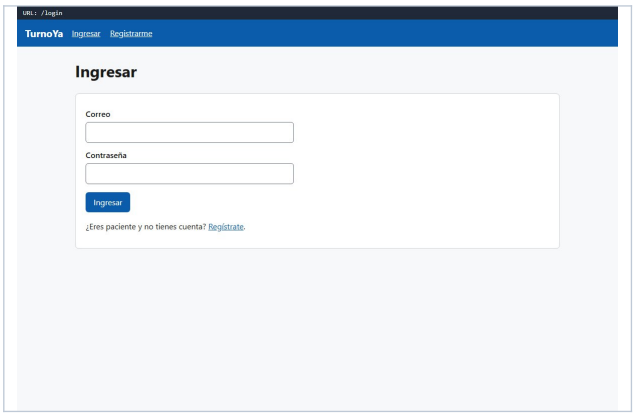


Figura 1. Inicio de sesión.

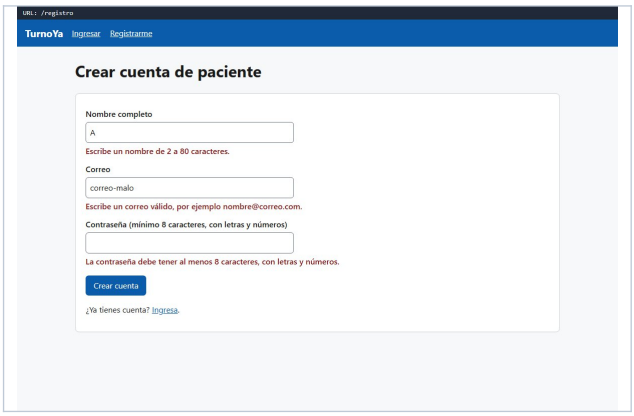


Figura 2. Registro con validación por campo.

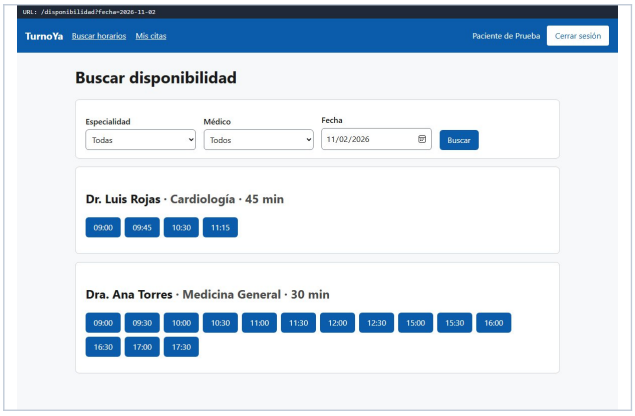


Figura 3. Búsqueda de disponibilidad del paciente.

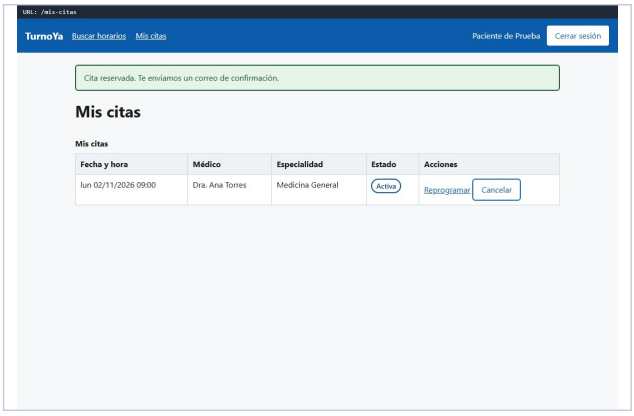


Figura 4. Reserva confirmada en «Mis citas».

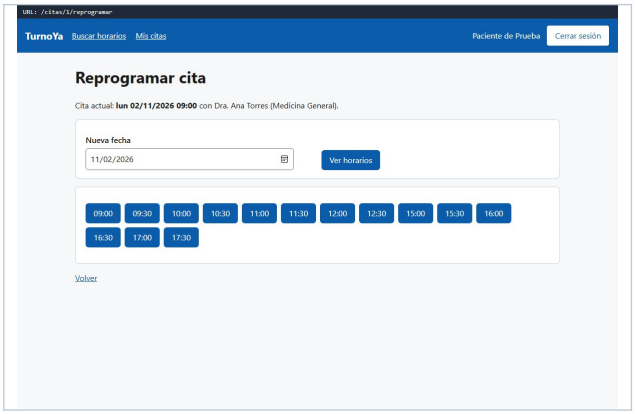


Figura 5. Reprogramación de una cita.

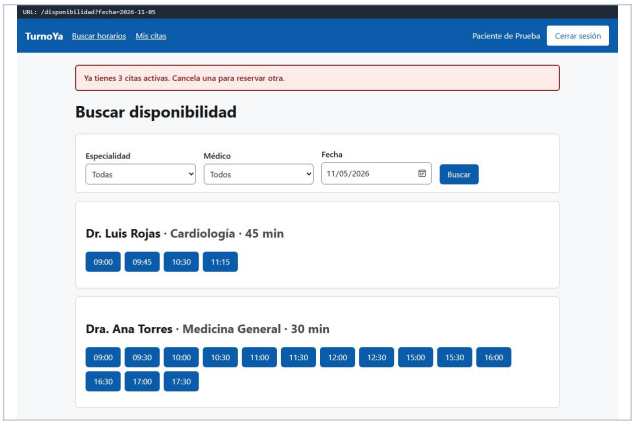


Figura 6. Rechazo por superar 3 citas activas.

5. Producto entregado (continuación)

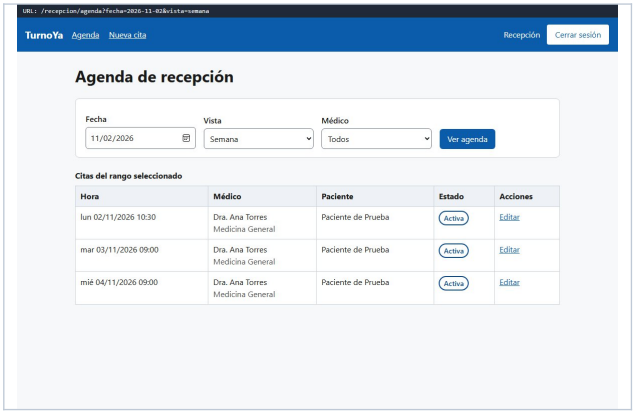


Figura 7. Agenda semanal de recepción.

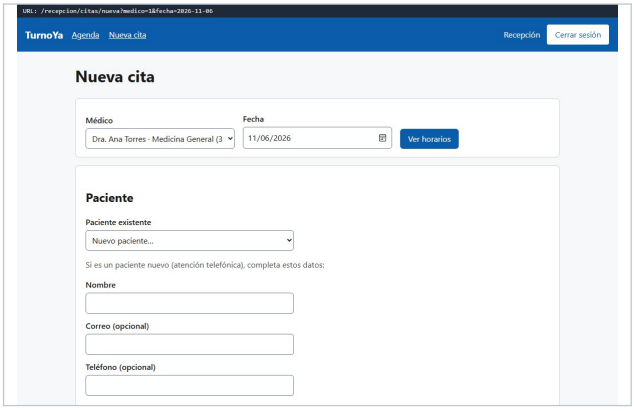


Figura 8. Cita registrada por recepción.

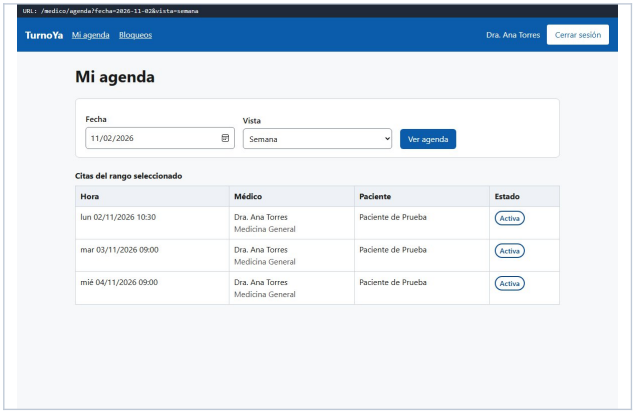


Figura 9. Agenda del médico.

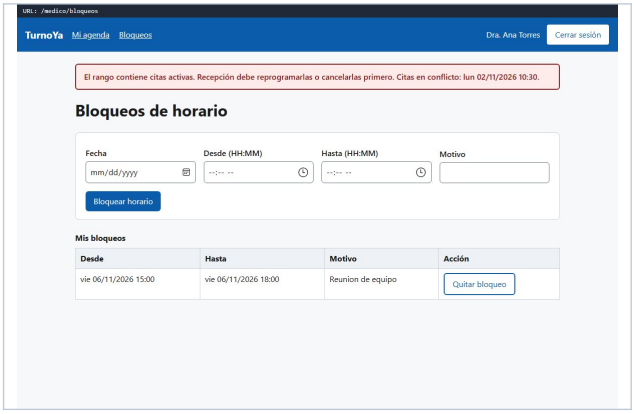


Figura 10. Bloqueo rechazado por citas en conflicto.

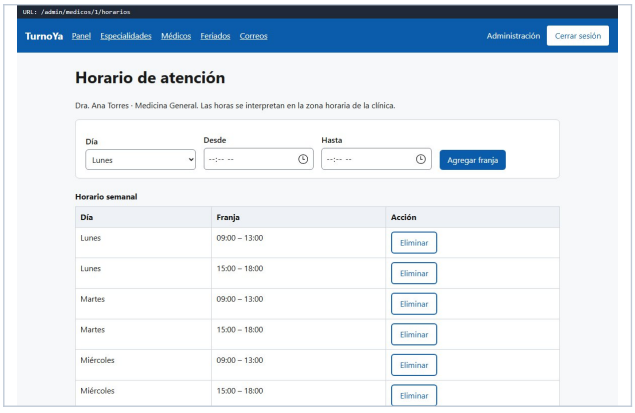


Figura 11. Horario de atención de un médico.

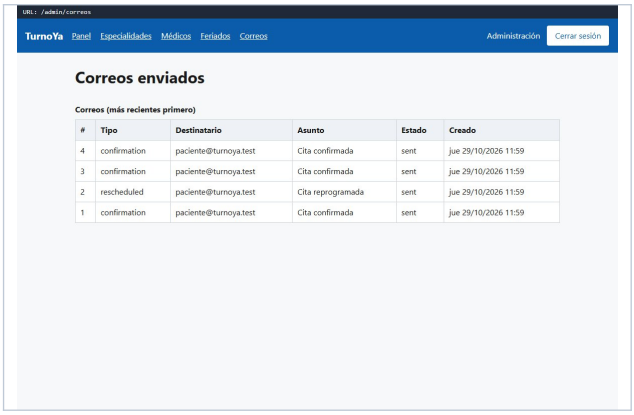


Figura 12. Bandeja de correos enviados.

6. Calidad y verificación

6.1 Estrategia de pruebas

Tipo de prueba	Cantidad	Qué verifica
Unitarias de dominio	48	Reglas de negocio en sus límites exactos, disponibilidad, zona horaria, validación de datos y contraste de colores.
Integración por historia	50	Flujos completos de cada rol sobre la aplicación real: registro, reservas, cambios, recordatorios, recepción, médico y administración.
Seguridad y accesibilidad	4	Protección CSRF en todos los formularios, cookies, cabeceras, registros sin datos personales y estructura accesible de 16 pantallas.
Concurrencia	2	12 pacientes reservando el mismo horario a la vez; restricción única en la base de datos.
Total	**104**	**104 pruebas aprobadas, 0 fallidas.**

6.2 Trazabilidad de reglas a pruebas

Regla	Requisito	Archivos de prueba
Duración 30 o 45 minutos	FR-004	availability, booking
Sin solapamiento por médico	FR-005	rules, booking, concurrency
Máximo 3 citas activas	FR-006	rules, booking, change
Anticipación mínima de 2 h	FR-007, FR-024	rules, booking
Cambios hasta 24 h antes	FR-008, FR-024	rules, change
Recordatorio único 24 h antes	FR-009	reminders
Bloqueo por 3 inasistencias	FR-012, FR-021	rules, reception
Bloqueos del médico	FR-013, FR-020	doctor
Administración y feriados	FR-014, FR-019	admin
Atribuciones de recepción	FR-022	rules, reception
Permisos por rol	FR-015	auth, change, a11y-security

6. Calidad y verificación (continuación)

6.3 Criterios de éxito

ID	Criterio	Estado	Evidencia o acción
SC-001	Registro y primera reserva en menos de 3 minutos	Pendiente	Medir con usuarios reales
SC-002	Ningún médico con citas solapadas	Cumplido	Pruebas de reglas y concurrencia
SC-003	Confirmación y un único recordatorio por cita	Cumplido	Pruebas de reserva y recordatorios
SC-004	Cero citas duplicadas en el primer mes	Pendiente	Medir en operación
SC-005	Agenda en < 5 s y cita en < 1 min	Pendiente	Medir con usuarios reales
SC-006	Reglas con pruebas de frontera aprobadas	Cumplido	104 pruebas aprobadas
SC-007	Todas las pantallas operables con teclado	Parcial	Falta prueba manual

Los criterios que dependen de reglas y garantías técnicas están demostrados. Los que dependen del uso real (tiempos y duplicados en operación) requieren medición durante la validación con la clínica.

6.4 Verificación de la eficacia de las pruebas

Para confirmar que las pruebas detectan errores reales, se modificó deliberadamente la regla de 24 horas a 12 horas. Fallaron exactamente las dos pruebas que cubren ese límite; tras restaurar la regla, la suite completa volvió a aprobarse.

6.5 Incidencias detectadas y corregidas

Incidencia	Detección	Corrección
Las tablas excedían el ancho de pantalla en teléfonos	Revisión en vista móvil	Contenedor con desplazamiento accesible
Los correos se enviaban con hasta un minuto de retraso	Revisión de la bandeja tras reservar	Envío inmediato tras cada operación, sin duplicados
Recordatorio inmediato al reprogramar a menos de 24 h	Revisión del diseño	Se considera la fecha de la última modificación

7. Limitaciones y próximos pasos

7.1 Limitaciones de la versión 1

Limitación	Impacto	Mitigación prevista
Envío de correo simulado	Los mensajes se guardan, pero no llegan al paciente	Conectar un servidor SMTP; la interfaz ya lo permite
Un solo proceso de servidor	No escala horizontalmente	Migrar a una base multiproceso si la demanda crece
Sin recuperación de contraseña	El paciente depende de recepción	Incluir en la versión 2
Accesibilidad verificada solo automáticamente	Riesgo con lectores de pantalla	Prueba manual con teclado y lector
Módulo SQLite de Node marcado como experimental	Posibles cambios futuros de la API	Acceso aislado en un único módulo

7.2 Próximos pasos recomendados

Orden	Acción	Responsable sugerido
1	Validar la versión 1 con recepción y dos médicos usando datos reales	Clínica y equipo de desarrollo
2	Conectar el envío real de correos y revisar los textos	Equipo de desarrollo
3	Medir SC-001 y SC-005 y realizar la prueba de accesibilidad manual	Equipo de desarrollo
4	Abrir la versión 2 con un nuevo ciclo SDD: recuperación de contraseña y mejoras de la validación	Equipo de desarrollo

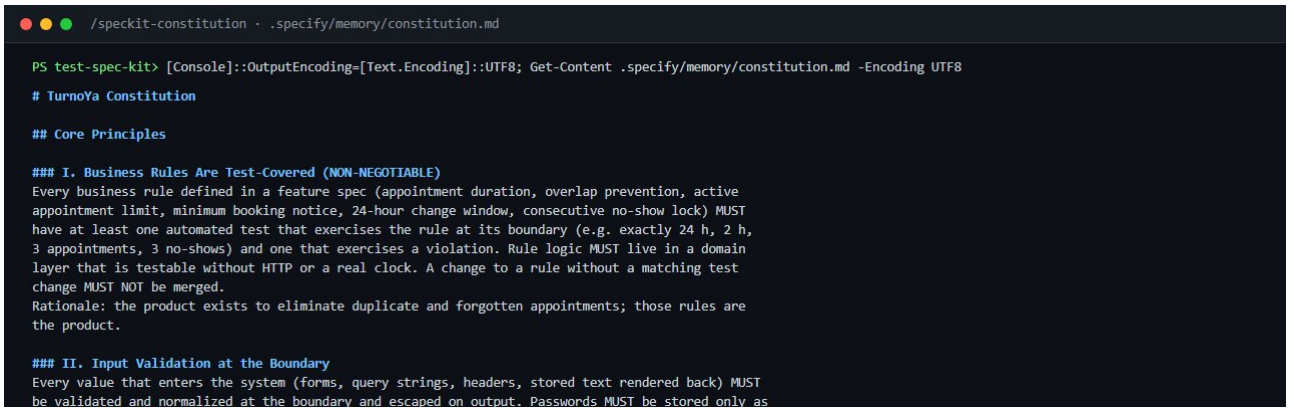
7.3 Conclusión

La versión 1 de TurnoYa cumple la especificación acordada: los pacientes pueden reservar y gestionar sus citas en línea, y la clínica aplica de forma automática las reglas que hoy dependen de la gestión manual. El proceso seguido deja trazabilidad completa entre requisitos, tareas, código y pruebas, lo que permite abordar la siguiente versión con la misma disciplina.

Anexo A · Evidencias del proceso SDD

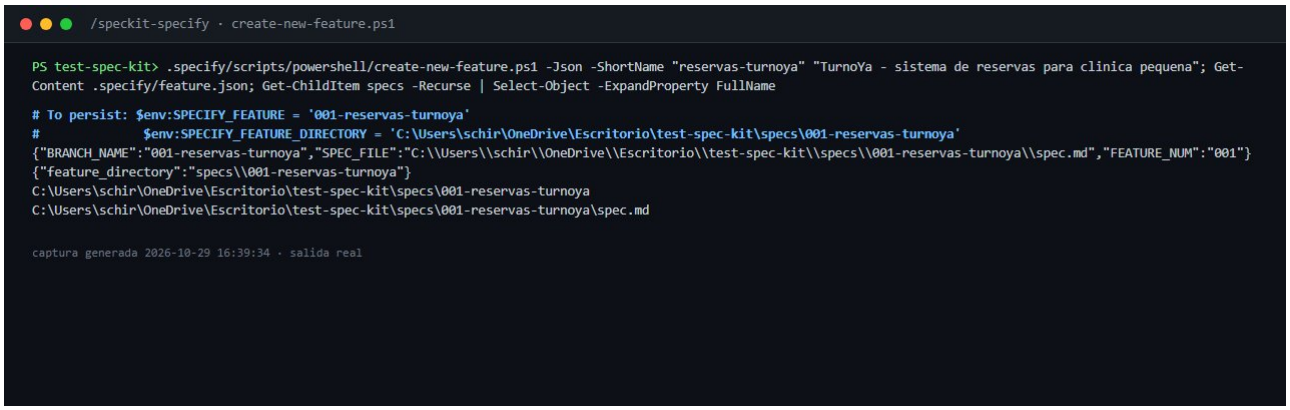
Selección de evidencias de cada fase. Los archivos completos están en la carpeta capturas/ del repositorio.

A.1 Constitución del proyecto



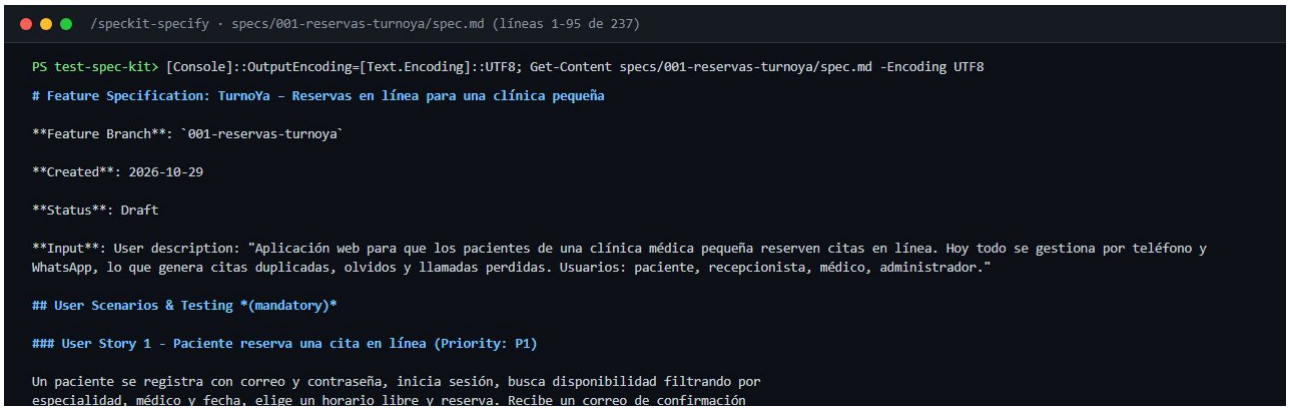
Principios obligatorios del proyecto, versión 1.0.0. Fuente: capturas/01-constitution/02-constitution-resultado.png

A.2 Creación de la funcionalidad



El script de Spec Kit asigna el número 001 y crea la carpeta de la especificación. Fuente: capturas/02-specify/01-create-new-feature.png

A.3 Especificación



Encabezado de spec.md y primera historia de usuario. Fuente: capturas/02-specify/02-spec-historias.png

Anexo A · Evidencias del proceso SDD (continuación)

A.4 Control de calidad de la especificación

/speckit-specify · checklists/requirements.md (validación de calidad)

```
PS test-spec-kit> [Console]::OutputEncoding=[Text.Encoding]::UTF8; Get-Content specs/001-reservas-turnoya/checklists/requirements.md -Encoding UTF8
# Specification Quality Checklist: TurnoYa – Reservas en línea para una clínica pequeña

**Purpose**: Validate specification completeness and quality before proceeding to planning
**Created**: 2026-10-29
**Feature**: [spec.md](../spec.md)

## Content Quality

- [x] No implementation details (languages, frameworks, APIs)
- [x] Focused on user value and business needs
- [x] Written for non-technical stakeholders
- [x] All mandatory sections completed

## Requirement Completeness

- [x] No [NEEDS CLARIFICATION] markers remain
- [x] Requirements are testable and unambiguous
```

Lista de verificación de Spec Kit con todos los puntos aprobados. Fuente: capturas/02-specify/04-checklist-calidad.png

A.5 Clarificación

/speckit-clarify · preguntas 1-3 (una por vez, con opción recomendada) (líneas 1-62 de 94)

```
PS test-spec-kit> [Console]::OutputEncoding=[Text.Encoding]::UTF8; Get-Content specs/001-reservas-turnoya/clarify-session.md -Encoding UTF8
# Sesión /speckit-clarify – 2026-10-29

> Las respuestas fueron elegidas aceptando la opción **Recomendada** de cada pregunta
> (el usuario pidió ejecutar el flujo SDD de forma autónoma). Cada respuesta se integró en
> `spec.md` (sección `## Clarifications` y requisitos afectados).

## Pregunta 1 de 5

**Question:** ¿En qué zona horaria se muestran y se evalúan todas las horas (reglas de 2 h / 24 h,
recordatorios, agenda del día)?
*Why it matters:* si se evalúa en la zona del dispositivo del paciente, el mismo horario tendría
límites distintos para cada persona y las reglas de 24 h serían inconsistentes.

**Recommended:** Option A - una sola zona horaria de la clínica evita ambigüedades en las reglas.

| Option | Description |
|--------|-------------|
```

Pregunta con opción recomendada y su justificación. Fuente: capturas/03-clarify/02-preguntas-1-3.png

A.6 Especificación clarificada

```
WhatsApp, lo que genera citas duplicadas, olvidos y llamadas perdidas. Usuarios: paciente, recepcionista, médico, administrador."

## Clarifications

### Session 2026-10-29

- Q: ¿En qué zona horaria se muestran y evalúan las horas? → A: Una zona horaria única, configurada para la clínica; todo se muestra y evalúa en ella.
- Q: ¿Qué ocurre con los días feriados? → A: El administrador registra feriados (fecha + nombre); ese día no se ofrece disponibilidad para ningún médico.
- Q: ¿Qué pasa si un médico bloquea un rango con citas activas? → A: El bloqueo se rechaza y se listan las citas en conflicto; recepción las reprograma/cancela primero.
- Q: ¿Cómo se cuentan las tres inasistencias consecutivas y cómo se levanta el bloqueo? → A: Consecutivas = las últimas 3 citas con resultado marcado son "no asistió"; una "asistió" reinicia el conteo a 0; las cancelaciones no cuentan; el paciente bloqueado puede asistir a citas agendadas por recepción y al asistir se desbloquea.
- Q: ¿Recepción está sujeta al límite de 3 citas y a las 2 h de anticipación? → A: Recepción está sujeta a duración y no-solapamiento, pero exenta del límite de 3 citas, de las 2 h de anticipación y del bloqueo por inasistencias.

captura generada 2026-10-29 16:41:43 · salida real
```

Decisiones incorporadas en la sección Clarifications. Fuente: capturas/03-clarify/04-spec-clarifications.png

Anexo A · Evidencias del proceso SDD (continuación)

A.7 Plan técnico

```
## Constitution Check

*GATE: Must pass before Phase 0 research. Re-check after Phase 1 design.*

| Principio | Cumplimiento en el plan | Estado |
|-----------|-------------------------|--------|
| I. Reglas de negocio con pruebas | `src/domain/rules.js` y `availability.js` puros; `tests/unit/rules.test.js` cubre cada regla en su frontera (24 h, 2 h, 3 citas, 3 inasistencias, solapamiento, duración) y su violación; tests escritos antes de la implementación (ver tasks) | PASS |
| II. Validación en el borde | `src/web/validate.js` valida/normaliza cada formulario; contraseñas con `scrypt` + sal; cookies `HttpOnly` + `SameSite=Lax`; verificación de rol y de propiedad en cada acción; HTML escapado por defecto en plantillas | PASS |
| III. Accesibilidad básica | HTML semántico, `label` ligados, errores en texto junto al campo, tablas con `th`, contraste AA, funciona sin JavaScript; verificación de teclado en tasks (fase Polish) | PASS |
| IV. Simplicidad / mínimas dependencias | 0 dependencias de terceros; una base; sin framework; alcance limitado a la spec | PASS |
| V. Tiempo explícito | `src/domain/time.js` es el único módulo que lee el reloj; UTC en base, `CLINIC_TZ` para decisiones; reloj inyectable en servicios y planificador | PASS |
| VI. Notificaciones idempotentes | tabla `outbox` con `dedupe_key` única; encolado dentro de la transacción de la cita, entrega posterior y reintentable; fallo de entrega no afecta la cita | PASS |

**Re-evaluación post-diseño (Phase 1)**: sin violaciones; `Complexity Tracking` vacío. Riesgo anotado:
`node:sqlite` aún emite aviso de "experimental" en Node 24 (ver research.md, decisión 2).
```

Comprobación de los seis principios de la constitución. Fuente: capturas/04-plan/02-plan-contexto-constitution.png

A.8 Modelo de datos

```
/speckit-plan · Phase 1: data-model.md

PS test-spec-kit> [Console]::OutputEncoding=[Text.Encoding]::UTF8; Get-Content specs/001-reservas-turnoya/data-model.md -Encoding UTF8

# Data Model: TurnoYa v1

Todos los instantes (`*_at`) se guardan como texto ISO-8601 UTC (`2026-11-02T15:00:00.000Z`).
Horarios de atención y feriados se guardan en hora local de la clínica (`HH:MM`, `YYYY-MM-DD`).

## Entidades

### users
| Campo | Tipo | Reglas |
|-------|------|--------|
| id | integer PK | |
| email | text | único, minúsculas, formato válido |
| password_hash | text | `scrypt$<sal>$<hash>`; nunca se muestra |
| name | text | 2–80 caracteres |
| role | text | `patient` \| `reception` \| `doctor` \| `admin` |
| doctor_id | integer FK null | obligatorio si `role='doctor'` |
```

Entidades, campos y reglas de validación. Fuente: capturas/04-plan/05-data-model.png

A.9 Lista de tareas

```
## Phase 1: Setup (Shared Infrastructure)

- [ ] T001 Crear `package.json` (type module, scripts `start`, `seed`, `test`; sin dependencias) y la estructura de carpetas
`src/{domain,db,services,web/{views,routes}}`, `tests/{unit,integration}`
- [ ] T002 [P] Crear `.gitignore` (`data/`, `node_modules/`, `*.log`, `.env*`) en la raíz
- [ ] T003 [P] Crear `src/config.js` con `PORT` (3000), `CLINIC_TZ` (America/Lima), `DB_PATH` (data/turnoya.db), `SESSION_TTL_HOURS` (8)

---

## Phase 2: Foundational (Blocking Prerequisites)

**⚠ CRITICAL**: ninguna historia puede empezar hasta terminar esta fase.

- [ ] T004 [P] Escribir pruebas de `time.js` en `tests/unit/time.test.js`: conversión local↔UTC en `America/Lima`, `addMinutes`, `FakeClock`, inicio/fin de día
local
- [ ] T005 Implementar `src/domain/time.js`: `Clock` (único módulo que lee `Date.now`), `FakeClock`, `zonedToUtc(date,time,tz)`, `utcToZoned`, `localDate(utc,tz)`,
`weekdayOf(date)` (Principio V, FR-018)
- [ ] T006 [P] Crear `src/db/schema.sql` con las tablas de data-model.md: users (`role` en patient|reception|doctor|admin), patients (`consecutive_no_shows` integer
≥ 0), specialties (`duration_min` 30 o 45), doctors, work_schedules (`start_time < end_time`), holidays, blocks (`starts_at < ends_at`), appointments (status
active|cancelled|attended|no_show; `UNIQUE(doctor_id, starts_at) WHERE status='active'`), outbox (`dedupe_key` UNIQUE), sessions
```

Tareas de preparación y base común con su formato obligatorio. Fuente: capturas/05-tasks/02-tasks-fases-1-3.png

Anexo A · Evidencias del proceso SDD (continuación)

A.10 Pruebas en rojo

```
/speckit-implement · TDD fase ROJA (tests escritos, sin implementación)

PS test-spec-kit> npm test 2>&1 | Select-Object -First 45

> turnoya@0.1.0 test
> node --disable-warning=ExperimentalWarning --test "tests/**/*.test.js"

node:internal/modules/esm/resolve:272
    throw new ERR_MODULE_NOT_FOUND(
          ^

Error [ERR_MODULE_NOT_FOUND]: Cannot find module 'C:\Users\schir\OneDrive\Escritorio\test-spec-kit\src\domain\availability.js' imported from
C:\Users\schir\OneDrive\Escritorio\test-spec-kit\tests\unit\availability.test.js
    at finalizeResolution (node:internal/modules/esm/resolve:272:11)
    at moduleResolve (node:internal/modules/esm/resolve:879:10)
    at defaultResolve (node:internal/modules/esm/resolve:1006:11)
    at #cachedDefaultResolve (node:internal/modules/esm/loader:705:20)
    at #resolveAndMaybeBlockOnLoaderThread (node:internal/modules/esm/loader:725:38)
    at ModuleLoader.resolveSync (node:internal/modules/esm/loader:763:56)
    at #resolve (node:internal/modules/esm/loader:687:17)
    at ModuleLoader.getOrCreateModuleJob (node:internal/modules/esm/loader:607:35)
```

Primera ejecución: las pruebas fallan porque el código aún no existe. Fuente: capturas/06-implement/02-tests-rojo-unitarias.png

A.11 Suite completa aprobada

```
✓ zonedToUtc respeta el horario de verano de otras zonas (1.0352ms)
✓ localDate y localTime devuelven la fecha/hora de la clínica, no la UTC (2.2577ms)
✓ weekdayOf, addDays y weekRange (1.6519ms)
✓ FakeClock permite fijar y avanzar el tiempo (0.36ms)
✓ registro válido normaliza el correo a minúsculas (2.8125ms)
✓ registro inválido devuelve un error por campo (0.3717ms)
✓ la contraseña exige letras y números (0.2633ms)
✓ isDate e isTime rechazan valores imposibles (0.4586ms)
✓ toInt devuelve NaN para texto no numérico (0.2792ms)
ℹ tests 104
ℹ suites 0
ℹ pass 104
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 5109.2224

captura generada 2026-10-29 17:02:13 · salida real
```

Resumen final: 104 pruebas aprobadas y 0 fallidas. Fuente: capturas/06-implement/05-tests-verde-suite-completa-2.png

A.12 Verificación de eficacia

```
Prueba de mutacion: se rompe la regla de 24 h a proposito y los tests deben fallar

PS test-spec-kit> [Console]::OutputEncoding=[Text.Encoding]::UTF8; $p = (Resolve-Path src/domain/rules.js).Path; $o = [IO.File]::ReadAllText($p);
[IO.File]::WriteAllText($p, $o.Replace("CHANGE_WINDOW_HOURS = 24","CHANGE_WINDOW_HOURS = 12")); "Mutacion aplicada: CHANGE_WINDOW_HOURS = 12"; try { npm test 2>&1 |
Where-Object { $_ -match "(tests|pass|fail) \d|24 h" } } finally { [IO.File]::WriteAllText($p, $o); "Regla restaurada: CHANGE_WINDOW_HOURS = 24" }

Mutacion aplicada: CHANGE_WINDOW_HOURS = 12
✖ a menos de 24 h no se puede reprogramar ni cancelar; a exactamente 24 h sí (FR-008, FR-024) (486.1497ms)
✓ recepción puede reprogramar y cancelar sin la ventana de 24 h (385.3764ms)
✓ el recordatorio se genera exactamente al entrar en la ventana de 24 h, una sola vez (FR-009, SC-003) (439.3498ms)
✓ reservada con menos de 24 h de anticipación: solo confirmación, sin recordatorio (371.4109ms)
✓ canModify: exactamente 24 h todavía se puede modificar (0.1738ms)
ℹ tests 89
ℹ pass 87
ℹ fail 2
✖ a menos de 24 h no se puede reprogramar ni cancelar; a exactamente 24 h sí (FR-008, FR-024) (486.1497ms)
  AssertionError [ERR_ASSERTION]: The input did not match the regular expression /hasta 24 horas antes/. Input:
    expected: /hasta 24 horas antes/,
Regla restaurada: CHANGE_WINDOW_HOURS = 24
```

Al alterar la regla de 24 h fallan sus pruebas; luego se restaura. Fuente: capturas/06-implement/06-mutacion-regla-24h.png

Anexo B · Ejecución del proyecto

Requisito: Node.js 24 o superior. No es necesario instalar dependencias.

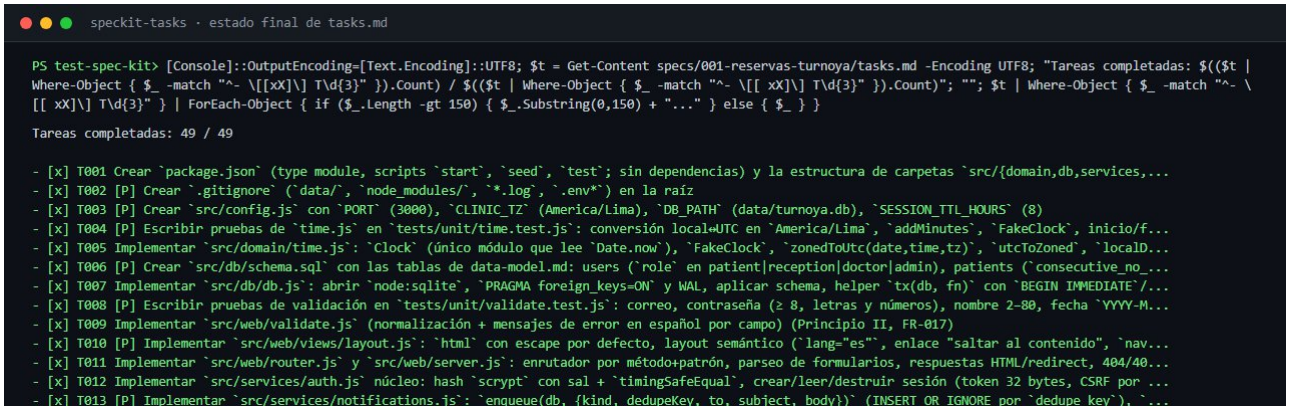
```
npm run seed     Crea la base de datos con datos de demostración
npm start        Inicia la aplicación en http://localhost:3000
npm test         Ejecuta las 104 pruebas automáticas
```

Cuentas de demostración

Rol	Correo	Contraseña
Administrador	admin@turnoya.test	Admin1234
Recepción	recepcion@turnoya.test	Recep1234
Médico	medico@turnoya.test	Medico1234
Paciente	paciente@turnoya.test	Paciente1234

Estas cuentas existen solo en la base de demostración y deben eliminarse antes de usar la aplicación con datos reales.

B.1 Estado final de las tareas



Las 49 tareas de tasks.md marcadas como completadas. Fuente: capturas/08-cierre/01-tareas-completadas.png

Anexo C · Inventario de artefactos

Ruta	Contenido
.specify/memory/constitution.md	Constitución del proyecto (v1.0.0)
specs/001-reservas-turnoya/spec.md	Especificación con clarificaciones
specs/001-reservas-turnoya/plan.md	Plan técnico y comprobación de la constitución
specs/001-reservas-turnoya/research.md	Decisiones técnicas y alternativas
specs/001-reservas-turnoya/data-model.md	Modelo de datos
specs/001-reservas-turnoya/contracts/	Contratos de rutas y de correos
specs/001-reservas-turnoya/tasks.md	Las 49 tareas, completadas
specs/001-reservas-turnoya/checklists/	Controles de calidad y accesibilidad
src/	Código fuente de la aplicación
tests/	Pruebas unitarias y de integración
capturas/	47 capturas del proceso y del producto
README.md	Guía de inicio rápido

Capturas por carpeta

Carpeta	Capturas	Contenido
00-setup	1	Estado inicial
01-constitution	2	Constitución
02-specify	4	Especificación
03-clarify	4	Clarificación
04-plan	6	Plan técnico
05-tasks	4	Tareas
06-implement	6	Implementación y pruebas
07-app-v1	18	Aplicación en ejecución
08-cierre	2	Cierre